Feature Engineering for Machine Learning

Project: music emotion recognition

Predict the emotion quadrant of a song from its audio, its lyrics, its album cover and its metadata.

The problem

Studies of how people search for and organise music single out its emotional content as an essential criterion [1]. Music emotion recognition (MER) predicts that content from the music itself. Interest in it has grown with music streaming platforms and their need to organise and recommend music to their users [1].

Emotion in music can be understood in three ways [1]: expressed, the emotion the composer or performer intends to convey; induced, the emotion evoked in the listener; perceived, the emotion the listener identifies in the song. This project addresses static MER, the identification of the predominant emotion perceived in a song [1]. One label per song, not one per second.

Emotion perception is inherently subjective [1]. Two listeners can hear the same song differently, so a label is the outcome of an annotation procedure, with errors of its own, not a measurement.

Emotion as a position

Emotions can be named as categories, or placed on continuous dimensions. Russell's circumplex model arranges affective concepts in a circle, on a plane defined by two bipolar dimensions, pleasure–displeasure and arousal–sleep [3]. In MER they are called valence and arousal. Dividing the plane at its centre gives four quadrants. They are the four classes of this project.

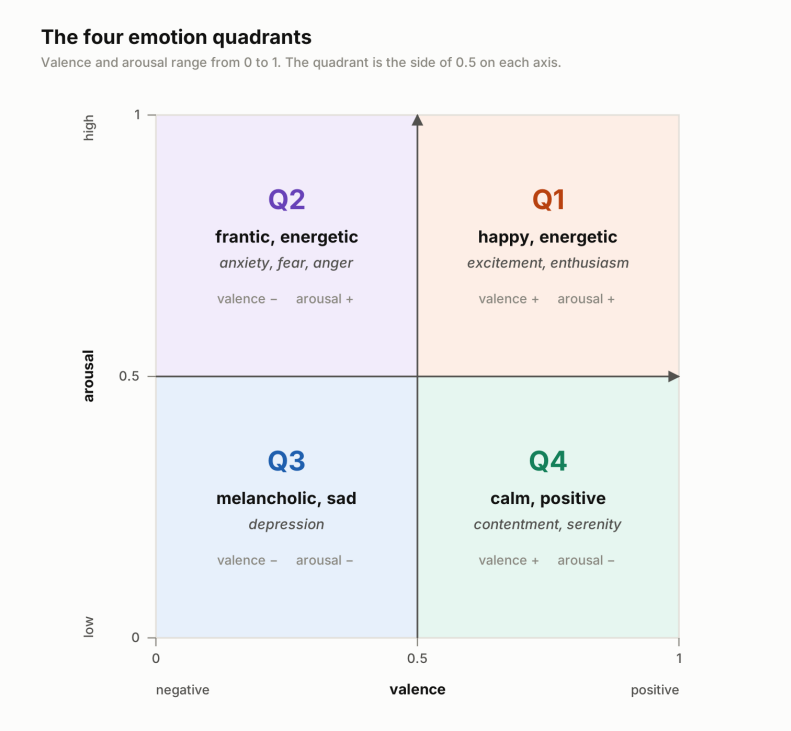


Figure 1. The four quadrants of the valence–arousal plane [3], with the descriptions and examples given by the authors of the dataset [1].

Every song comes with a valence and an arousal value between 0 and 1, in `av_values.csv`. Its quadrant is the side of 0.5 on each axis. These two values are the target in continuous form, not an input.

MERGE

MERGE [1, 2] is a dataset for static MER, released in 2024 by researchers of the Centre for Informatics and Systems of the University of Coimbra (CISUC). It contains three sets, audio, lyrics and bimodal, each in a complete and a balanced version. The bimodal set keeps only the songs whose audio and lyrics were annotated in the same quadrant [1]. Its complete version, 2216 songs, is the one used here.

The labels were obtained semi-automatically [1]. Emotion tags from AllMusic were mapped onto valence and arousal with the norms of Warriner et al. for 13,915 English lemmas [4]. Songs whose position was ambiguous were discarded. The rest were validated manually by eight people, audio and lyrics separately.

To the best of its authors' knowledge, MERGE is the largest publicly available and quality-controlled bimodal static MER dataset [1]. It is distributed under a CC BY-NC 4.0 licence [2], with two predefined train, validation and test partitions, 70-15-15 and 40-30-30, provided to make results reproducible [1].

Its authors report that lyrics are better at capturing valence, and audio at capturing arousal [1]: each modality sees part of the plane. The best result they report on the bimodal complete set is an F1-score of 81.74 %, with a neural network combining audio and lyrics, on the 70-15-15 partition [1]. That number is a reference, not a target. The data, the partition and the classifier here are different.

The approach

The version of MERGE used here (**1**) adds the album covers, damages the metadata tables on purpose and alters the class distribution. Cleaning it is the first half of the work.

The classifier is fixed: **k nearest neighbours**. Everything that decides the result happens before it.

From **7** onwards the classifier is not the object of study, it is the instrument. Each decision in the pipeline is a hypothesis, tested by running the same classifier with one thing changed. There are five such tests. The first one chooses how every other one is measured.

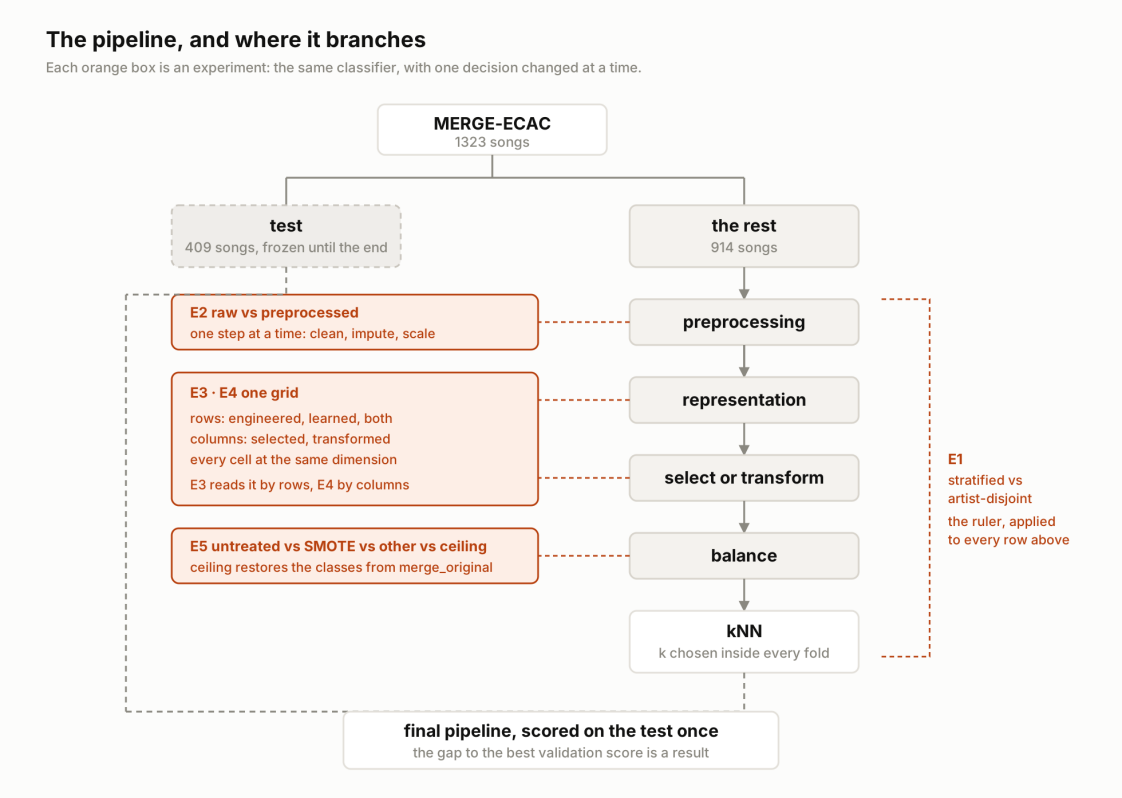


Figure 2. The pipeline, and where it branches.

Work and delivery

Groups of three. One delivery, on Inforestudante, until 23:59 on Saturday, 5 December 2026. The rules of assessment are in **18**.

The questions are numbered. They are answered in a report, each answer under the number of its question. The suggested outputs indicate what usually supports an answer. A figure or a table is not an answer without the sentence that reads it.

The code is written as Python files or as Jupyter notebooks. A group working in notebooks chooses one of two forms:

- **the notebook as the report.** The answers are written in Markdown cells, under the number of each question, next to the code that supports them. Exploration that supports no answer is left out. The notebook is delivered with its outputs.
- **a separate report.** The notebooks are delivered as code, alongside it.

In either form, every number and every figure in the report is produced by the code delivered, and the code runs from start to end in a clean environment.

Calendar

date	event
5 October 2026	brief published
19 October to 27 November 2026	continuous assessment, six weeks. In each PL class, half of the groups hold their three checkpoints in weeks 1, 3 and 5 (from 19 October, 2 November and 16 November), the other half in weeks 2, 4 and 6 (from 26 October, 9 November and 23 November)
23 October 2026	first mini-test, in the TP class
20 November 2026	second mini-test, in the TP class
Saturday, 5 December 2026, 23:59	final delivery, on Inforestudante
last two PL classes of the semester	project defences
normal or resit exam season	third mini-test, at the student's choice

Proposed work schedule

A suggested pace, not a requirement: the nine weeks from the publication of the brief to the final delivery, and the sections to close in each.

week	dates	sections
1	5 to 11 October	1. The dataset, 2. Data understanding, 3. Exploratory analysis
2	12 to 18 October	4.1 Duplicates, 4.2 Missing values, 4.3 Errors
3	19 to 25 October	4.4 Outliers, 4.5 Media, 4.6 The cleaned dataset, 5. Class distribution
4	26 October to 1 November	6. Evaluation of the cleaning, 7. The classifier, 8. Validation protocol
5	2 to 8 November	9. Effect of the preprocessing, 10.1 engineered features
6	9 to 15 November	10.2 to 10.6 learned representations, 11. Selection and transformation
7	16 to 22 November	12. Representation against treatment
8	23 to 29 November	13. Class balancing, 14. Final model and test
9	30 November to 5 December	15. Interpretation, 16. Deployment, 17. Deliverables

1. The dataset

Derived from MERGE Bimodal Complete [1, 2], extended with album covers, and deliberately damaged.

path	content
`audio/`, `lyrics/`, `album_covers/`	the media, one folder per quadrant
`merge_ecac/`	the damaged tables. All work is done on these.
`merge_original/`	the tables as published
`truth/`	every modification made, per cell and per row
`DATA_DICTIONARY.md`	what each column was meant to contain
`IMBALANCE.md`	what was done to the class distribution

The original dataset is public and is therefore provided. `truth/` is not an answer sheet; it is an instrument, used in **6**. A cleaned dataset delivered without those measurements is not accepted.

The media files were not modified. The tables were. The original has defects of its own, recorded in no file.

The test partition. `merge_ecac/test_songs.csv` lists 409 songs, 31 % of the dataset, from the partition published with MERGE. Separate them before anything else. Nothing may be fitted on them: no scaler, no imputation, no vocabulary, no selection, no hyperparameter. They are cleaned like the rest and then left untouched until **14**.

Provided during the project. Two further resources will be made available:

- the pre-trained models for the learned representations of **10**, one per domain that has one;
- a base application for the deployment of **16**.

The models provided are a starting point, not an obligation: other pre-trained models may be used instead (**10.2**).

2. Data understanding

2.1 What is each column supposed to contain? Which are measurements, which are identifiers, which are administrative?

2.2 Do the four sources join one to one? If not, where does it break, and what follows from that?

2.3 From the documentation alone, before computing anything: which columns are suspected of giving away the target? Record the list.

Suggested outputs

- the list of **2.3**, dated before any analysis

3. Exploratory analysis

3.1 Describe every column: distribution, range, missing values.

3.2 Which columns are nearly empty? Which are constant? Which measure something other than what their name says?

3.3 Measure how well each column alone predicts the quadrant. Some do it far too well. Which, and by what mechanism did the target reach them?

3.4 Compare **3.3** with the list of **2.3**. What proportion was obtainable by reading alone?

3.5 How many songs does each artist have? How many test songs share an artist with the rest of the data? Required in **8**.

Suggested outputs

- a table of every column: type, missing rate, association with the target
- the list of columns to discard, with the reason
- a histogram of each numeric column
- a matrix of pairwise scatter plots
- a sorted bar chart of the association of each column with the target

4. Cleaning

4.1 Duplicates

4.1.1 Identify the identical rows.

4.1.2 No column states which rows are the same recording. Compare four kinds of evidence: the identifier columns, the artist and title, the lyrics, the audio. Do they agree? Which of them establishes that two rows are the same recording?

4.1.3 A remix, an edit or a live version is the same song and a different recording. Is it a duplicate?

4.1.4 Does any recording, or any song, appear on both sides of the test boundary? What does that do to the test score, and what can be done about it without changing the test partition?

4.1.5 How many artists are written in more than one way? What does that do to a cross-validation that places all the songs of each artist in the same fold?

4.1.6 What should be done with each kind of duplicate? Justify.

Suggested outputs

- a table of duplicate groups by kind, with the evidence that established each
- the recordings and songs found on both sides of the test boundary

4.2 Missing values

4.2.1 Classify the missingness mechanism of each column. Justify from the data, not from the column name.

4.2.2 Impute. Why should the method differ between columns?

4.2.3 One column is missing in a way that makes a particular imputation both tempting and wrong. Which, and why?

Suggested outputs

- the mechanism assigned to each column, with its evidence
- a map of missing values across rows and columns, ordered to show the pattern

4.3 Errors

A value that cannot be true. Found by a rule, with no statistics.

4.3.1 Identify the impossible values: sentinel codes, values outside the domain of the column, dates that have not happened.

4.3.2 Identify the values that are impossible only next to another column, where each one alone is unremarkable. Which rule finds them?

Suggested outputs

- a table of the rules applied and how many values each one caught

4.4 Outliers

A value that is unusual. It may be perfectly true.

4.4.1 Detect outliers with univariate methods. Compare the z-score, the MAD-based modified z-score and Tukey's fences. Which worked better, on which columns, and why do they disagree?

4.4.2 Detect outliers with multivariate methods. Compare k-means and DBSCAN on a numeric feature space. Justify the parameters of each.

4.4.3 Which of the two separates a record that is alone from a small group of records that are together and far from everything else? What does that distinction change in the decision to delete?

4.4.4 At least one record is unremarkable in every column, violates no rule, and is still isolated. Which method found it, and which did not?

4.4.5 Which of the flagged records are extreme and correct, and must survive the filter? How does the criterion keep them?

4.4.6 Errors and outliers are different things. From this dataset, give one example of each: an error that is also an outlier; an error that no univariate method flags and a rule over two columns does; an error that no rule finds and only the joint distribution reveals; an outlier that is not an error.

Suggested outputs

- a table of how many values each univariate criterion flags, per column
- a box plot per numeric column with Tukey's fences, and the values flagged by each of the three univariate criteria marked differently
- the curves used to choose the parameters: the distance to the k-th neighbour for DBSCAN, and the elbow or silhouette for k-means
- the k-means and DBSCAN solutions side by side, in the same two dimensions, with the records each one flags marked
- the record of **4.4.4**, shown in the columns where it stands out
- the four examples of **4.4.6**

4.5 Media

4.5.1 Inspect the covers and the audio. Repeated images, missing covers, excerpts that are not what they claim. Does any of it threaten the evaluation?

Suggested outputs

- a scatter plot of cover width against height
- the distribution of each audio statistic over the corpus

4.6 The cleaned dataset

4.6.1 Apply the decisions of **4.1** to **4.5** and produce the cleaned dataset. The test partition is cleaned by the same rules, fitted on the rest.

4.6.2 What did each decision cost? Rows removed, values replaced, information lost.

Suggested outputs

- the cleaned dataset

- a table of decisions, one row per decision, with its cost

5. Class distribution

5.1 Report the distribution and the ratio between the largest and smallest class.

5.2 Which metrics stop being informative at that ratio?

5.3 What does a majority-class classifier obtain?

Read `IMBALANCE.md` before **13**.

Suggested outputs

- the three numbers above
- a bar chart of class frequencies, before and after cleaning

6. Evaluation of the cleaning

6.1 Using `truth/`, evaluate the detectors of **4** as classifiers. What was found, what was missed, what was flagged and had never been damaged?

6.2 Take ten records flagged by a detector and absent from `truth/`. For each: an error, a legitimate extreme, or a member of a real sub-population?

6.3 Compare imputation methods on one column against the real values.

6.4 What do these numbers allow one to conclude about the columns for which no ground truth exists?

Suggested outputs

- found and missed, per defect family
- the verdict on each of the ten records of **6.2**
- a bar chart of recall by defect family
- a precision-recall curve with the chosen operating point marked

7. The classifier

7.1 Build the simplest pipeline that runs end to end: the numeric columns of `metadata.csv` as read, median imputation, no scaling, kNN, five-fold cross-validation stratified by class, on the 914 songs outside the test partition.

7.2 Run it with k = 5, then with k chosen properly. Report both, as accuracy and macro F1. Where they disagree, why?

A condition. Every comparison from here on is between two conditions. A condition is never a pipeline with a hyperparameter frozen in it; it is the best that pipeline achieves:

- hyperparameters are chosen inside each fold, on the training part of that fold alone, and the chosen model is scored on the part held out;
- the same grid is used in every condition. At minimum: k over the odd values from 1 to 31, two distance functions, uniform and distance weighting;
- everything fitted from data is fitted inside the fold. Imputer, scaler, encoder, vocabulary, reduction, selection, resampling;
- the score is macro F1, as the mean over folds with its spread. Accuracy is reported alongside.

7.3 Why does choosing k on the same folds that are then reported produce a number that does not survive the test partition?

7.4 When a change to the pipeline moves the best k, what does that say about the feature space?

Suggested outputs

- the baseline score
- the hyperparameters chosen by every condition reported from here on

8. Validation protocol

> **Experiment 1.** The same pipeline, scored two ways: five-fold cross-validation stratified by class, and five-fold cross-validation with all the songs of each artist in the same fold, so that no artist appears on both sides of a split (artist-disjoint).

8.1 Report both estimates and the gap.

8.2 The two estimates answer two different questions. Which question does each answer?

8.3 The test partition was not built by artist. Given **3.5**, which of the two estimates should it reproduce, and why?

8.4 Which of the two questions will the application of **16** face? Which protocol is used for the rest of the project? Justify.

Suggested outputs

- both estimates and their gap
- the protocol used from here on

9. Effect of the preprocessing

> **Experiment 2.** The pipeline of **7**, adding one step at a time: raw; cleaned; cleaned and imputed per column; cleaned, imputed and scaled. Every step tuned as **7** requires.

9.1 Report the score after each step.

9.2 Which step moved the score most? Why is that step decisive for kNN in particular?

9.3 Does the order of the steps matter? Give a case from this dataset where it does.

9.4 Repeat the cleaning step once per defect family of **4**. Which part of the cleaning paid for itself?

9.5 If a step lowered the score, what does that mean, and is it kept anyway?

Suggested outputs

- a table with one row per step and one row per defect family: score, difference, chosen k

10. Representations

10.1 Extract engineered features from each domain. The choice of features and of tools is free. Audio, lyrics and cover have at least 10 columns each, covering at least:

domain	groups covered
audio	4 of: melody, harmony, rhythm, dynamics, tone colour, expressivity, texture, form [5]
lyrics	3 of: content, style, song structure, semantics [6]
cover	3 of: colour, texture, composition, content [7]

The metadata contributes the columns kept in **3**.

A feature is one column: the mean and the standard deviation of a descriptor are two. Each feature is documented with what it measures, how it is computed and with which parameters, and the axis of Figure 1 it is expected to inform: valence, arousal or neither. Features computed the same way are documented together. Does the data support the expected axis? Compare it with the association of each feature with valence and with arousal, on the training songs.

10.2 Extract learned representations with pre-trained models, at least one for each of audio, lyrics and cover. The models provided may be used, or others chosen by the group. A model chosen by the group is identified by name, version and source, is used without further training on this dataset, and must run in the base application of **16**.

10.3 What was each model trained to do? How far is that task from recognising emotion?

10.4 Each model expects its input in a particular form. What does each one expect, and what had to be done to the audio, the lyrics and the covers to meet it? What was lost in the process?

10.5 The tag columns of the metadata are lists. How are they encoded, and which of them can be used at all?

10.6 How was it guaranteed that nothing was fitted on the test partition?

Suggested outputs

- the feature table of **10.1**: one row per feature or group of features, with its domain, group, computation and expected axis
- one matrix per domain and per family, with its dimension and the cost of producing it
- for each model used, the input it expects and what was done to the data to meet it
- the distributions by class of the features that separate best
- a correlation matrix per domain

11. Selection and transformation

Two ways of reducing a representation. **Selected**: a subset of the original features. **Transformed**: new features built from all of them.

11.1 Select with a filter, a wrapper and an embedded method, on the engineered features of the training songs. Do they agree? Which features survive all three?

11.2 The embedded method ranks features with a model other than kNN. Does that matter?

11.3 One selection method is carried into **12**, where it is fitted again inside every fold of every cell. Which one, and why? What would the wrapper cost there?

11.4 Transform with PCA and with LDA. How many dimensions can LDA produce with four classes? Which of the two uses the labels, and what does that require of the protocol?

11.5 What does a single dimension of a learned embedding mean? What follows for selecting among them?

11.6 kNN weighs every dimension equally. With about 700 songs to train on, what happens to the distances between songs as the number of dimensions grows? Why, then, is no representation compared at its native dimension?

Suggested outputs

- the features kept by each selection method, and their overlap
- every representation of **10** in selected and in transformed form, at four or more target dimensions
- explained variance against number of components
- the data projected to two dimensions by PCA and by LDA, coloured by class

12. Representation against treatment

> **Experiments 3 and 4.** One grid. Every cell is a condition, and every cell has the same dimension, for four or more values of it, three among them. LDA cannot go beyond three, so its column is filled at three only.

	selected	PCA	LDA
engineered			
learned			
both			

> Experiment 3 reads the grid by rows: engineered against learned, under the same treatment. Experiment 4 reads it by columns: selected against transformed, and PCA against LDA, for the same representation.

12.1 Which representation wins, under which treatment, at which dimension? Does the winner change with the treatment?

12.2 Does the best treatment depend on the representation? Why might it?

12.3 How few dimensions can be kept before the score moves?

12.4 Joining two representations lets the larger one dominate the distance, and the variance that PCA sees. How were they joined, and how was that prevented?

12.5 Which distance suits each representation, and why?

12.6 Repeat the best cell once per domain. Which domain carries the signal?

12.7 Report the cost of each cell. At what price is the cheaper option still preferable?

Suggested outputs

- the grid, at every dimension tested
- a plot of score against dimension, one line per cell
- the cell carried into the final pipeline

13. Class balancing

> **Experiment 5.** Four conditions: untreated, SMOTE, one other remedy, and the training data with the subsampled classes restored from `merge_original/` and cleaned like the rest, which is the ceiling. Nothing is restored into the test partition.

13.1 What proportion of the gap to the ceiling does each remedy recover?

13.2 The two subsampled classes are equally rare and do not behave the same way. Explain the difference, per class and not on the average.

13.3 SMOTE builds its samples with nearest neighbours too. What does that imply for a classifier that then looks for nearest neighbours?

13.4 Resampling applied in the wrong place inflates the result. Demonstrate it.

Suggested outputs

- a table of the four conditions, with per-class recall
- the synthetic samples among the real ones, in two dimensions

14. Final model and test

14.1 Collect the five experiments in one table.

experiment	condition A	condition B	score A	score B	difference	hyperparameters	kept

For experiments 3 and 4, one row per reading of the grid: the best row against the next, and the best column against the next.

14.2 Build the final pipeline from the decisions that earned their place. Choose its hyperparameters with the protocol of **8** on the 914 songs, refit on all of them, and score once on the test partition.

14.3 Report the test score with an interval.

14.4 Compare it with the best validation score. Three things separate them: the choice of the best among many conditions, the artists shared between the test partition and the training data, and the size of the test partition. In which direction does each push, and which dominates here?

14.5 Report per-class precision, recall and F1, and a confusion matrix. Which pair of classes is confused, and would a person confuse the same pair?

14.6 Compare against the majority-class baseline, against **7**, and against the published results for this dataset. Is that last comparison legitimate?

14.7 The smallest class has 41 songs in the test partition. What do the per-class intervals permit concluding?

Suggested outputs

- the table of **14.1**
- the test score with its interval, and the analysis of **14.4**
- a confusion matrix
- score against k, for the baseline and the final pipeline on the same axes
- a single chart of the five experiments

15. Interpretation

15.1 Which features does the model use? Use a model-agnostic method.

15.2 Explain two songs classified correctly and two classified incorrectly, through their nearest neighbours.

15.3 Does the model agree with **3**? If a feature identified there as leakage is still present, find it.

Suggested outputs

- the ranking of **15.1**, as a chart showing the direction of each effect where it exists
- the four explanations of **15.2**

16. Deployment

16.1 Integrate the final pipeline into the base application provided. Input: metadata, an audio file, a lyrics file, a cover image. Output: the predicted quadrant and a confidence.

16.2 What does the base application expect from a pipeline? What had to change in the pipeline to meet it?

16.3 Every transformation of the training pipeline runs again here, in the same order, with the same fitted parameters. Demonstrate it: ten songs from the test partition through the application, predictions identical to those of the notebook.

16.4 Which of the metadata fields used by the model would exist for a song that is not yet in any catalogue? What follows for the model?

16.5 What does the application do when a field is missing? Justify.

Suggested outputs

- the base application running the final pipeline
- the table of **16.3**: ten songs, two predictions each

17. Deliverables

The suggested outputs of each section are a guide, not a checklist. What is required is the following:

- the report, answering the numbered questions and containing the grid of **12** and the table of **14.1**: a document of at most twelve pages, or the notebooks themselves, in the form described under *Work and delivery*;
- the code, as Python files or notebooks, with its dependencies listed, runnable from start to end in a clean environment with fixed random seeds;
- the base application with the final pipeline integrated, and instructions to run it.

18. Assessment

The practical component. The project and the mini-tests form the practical component of the course: 40 % of the final grade, 8 out of 20. The project is worth 5, the mini-tests 3. The minimum is 35 % of the component, the project and the mini-tests taken together.

The defence. Individual and mandatory. Its result, a percentage, multiplies the mark of the project. Each student reaches it in one of two ways:

- continuous assessment: three checkpoints in the PL classes, worth 25 % of the defence each, and the final defence for the remaining 25 %;
- the final defence alone, for a student who does not take the checkpoints.

The checkpoints can be recovered: the first and the second at the following checkpoint, the third at the final defence.

The mini-tests. Three, multiple choice, 20 minutes each. The best two count. Wrong answers are penalised, blank answers are not. The first two are held in TP classes on fixed dates. The third is held in the exam season, normal or resit, at the student's choice; a student who takes it in the normal season cannot repeat it in the resit season.

Additional oral defence. At any moment, any student may be asked for an individual oral defence on any component of the assessment. Its result prevails over the mark previously obtained in that component.

The mark of the project.

criterion	weight
Data preparation and its justification	20%
Features, representations, selection	20%
Experimental protocol and honest evaluation	20%
Interpretation and critical reading of results	20%
Deployment	10%
Report and code	10%

Marks are earned by the reasoning, not by the accuracy. A decision that lowered the score and was kept for a good reason is worth more than one that raised it and cannot be explained. Two groups with the same final score do not necessarily receive the same mark.

Any result obtained by fitting on the test partition, at any stage, voids the section it belongs to.

Use of AI in the preparation of this brief

A generative AI model, Claude Opus 5.5, was used in two parts of the preparation of this brief: to create the damaged version of the dataset described in 1, and to make textual adjustments to this document. All of its content is the responsibility of the teaching staff of the course.

References

[1] P. L. Louro, H. Redinho, T. F. R. Ribeiro, R. Santos, R. Malheiro, R. Panda, R. P. Paiva, “MERGE: A Bimodal Audio-Lyrics Dataset for Static Music Emotion Recognition”, *IEEE Transactions on Affective Computing*, vol. 17, no. 2, pp. 2058–2075, 2026. https://doi.org/10.1109/TAFFC.2026.3668037

[2] P. L. Louro, H. Redinho, R. Santos, R. Malheiro, R. Panda, R. P. Paiva, “MERGE Dataset”, version 1.1.0, Zenodo, 2024. https://doi.org/10.5281/zenodo.13939205

[3] J. A. Russell, “A circumplex model of affect”, *Journal of Personality and Social Psychology*, vol. 39, no. 6, pp. 1161–1178, 1980. https://doi.org/10.1037/h0077714

[4] A. B. Warriner, V. Kuperman, M. Brysbaert, “Norms of valence, arousal, and dominance for 13,915 English lemmas”, *Behavior Research Methods*, vol. 45, no. 4, pp. 1191–1207, 2013. https://doi.org/10.3758/s13428-012-0314-x

[5] R. Panda, R. Malheiro, R. P. Paiva, “Audio Features for Music Emotion Recognition: A Survey”, *IEEE Transactions on Affective Computing*, vol. 14, no. 1, pp. 68–88, 2023. https://doi.org/10.1109/TAFFC.2020.3032373

[6] R. Malheiro, R. Panda, P. Gomes, R. P. Paiva, “Emotionally-Relevant Features for Classification and Regression of Music Lyrics”, *IEEE Transactions on Affective Computing*, vol. 9, no. 2, pp. 240–254, 2018. https://doi.org/10.1109/TAFFC.2016.2598569

[7] J. Machajdik, A. Hanbury, “Affective image classification using features inspired by psychology and art theory”, *Proceedings of the 18th ACM International Conference on Multimedia*, pp. 83–92, 2010. https://doi.org/10.1145/1873951.1873965